PROJECT DOCUMENTATION & REPORT

Project Title: AI-Powered Support Ticket System

Implementation Stack: Next.js (App Router), Tailwind CSS, Full-Stack API Routing, Generative AI Integration

1. Executive Summary

This project presents a functional, modernized full-stack web application designed to optimize customer support workflows. By utilizing server-side rendering via Next.js and high-contrast, scalable styling with Tailwind CSS, the platform allows users to submit support requests that are dynamically handled, categorized, and monitored. The core engine bridges traditional form-based problem log entries with intelligent triage data points to optimize issue resolution times.

2. Problem Statement

Traditional customer relationship management (CRM) and helpdesk operations suffer from major inefficiencies:

- **Manual Triage Delays:** Incoming tickets are often manually sorted, leading to delays in critical bug identification.
- **Lack of Clear Prioritization:** Critical server or transactional issues get lost in logs behind low-priority feature requests.
- **Operational Burnout:** Support agents spend excessive time manually assessing context and parsing raw user input before drafting responses.

3. Technology Stack & Justification

Component	Technology	Selection Justification
Frontend Framework	Next.js (App Router)	Enables seamless client-side page routing, server-side performance improvements, and organized folder layout structures.
Styling Engine	Tailwind CSS	Utilizes atomic utility classes to build a fully accessible, professional design with explicit high-contrast text visibility.

Component	Technology	Selection Justification
Backend & APIs	Next.js API Routes	Provides a lightweight full-stack runtime capable of handling incoming JSON payloads and routing state updates securely.
State Tracking	Native JavaScript Hooks	Ensures predictable state tracking for interactive elements (forms, status filters, and interactive dashboard views).

4. Key Features & Functionality

- **Dynamic Ticket Dashboard:** A main workspace where technical support agents can view, filter, and track all submitted issues across their lifecycles.
- **Structured Priority Triage:** Tickets are classified explicitly using three priority tiers: Low, Medium, and High to ensure urgent bugs get immediate visibility.
- **Lifecycle State Tracking:** Allows operational updates across states: Open, In-Progress, and Resolved.
- **Robust Form Handling:** Client-side forms with comprehensive error-handling to prevent the submission of blank or malformed data into the pipeline.
- **Intelligent API Layer:** A backend routing hook prepared to connect with Generative AI capabilities to read user entries, analyze problem context, and automate categorization.

5. Local System Architecture & Installation Setup

The application is fully configured for a local development lifecycle.

Prerequisites:

- Node.js (v22.x or higher)
- npm (Node Package Manager)

Step-by-Step Local Deployment Setup:

1. **Initialize Dependencies:**

Bash

npm install

2. **Execute Development Server:**

```bash

npm run dev

3. **Local Access URL:** Open your web browser and navigate to http://localhost:3000 to interact with the operational portal.

**6. Future Enhancements**

- **Advanced Relational Schemas:** Integrating an external SQL database layer to run transactional analytical queries over historical ticket velocities.
- **Multi-Agent Workflows:** Implementing autonomous multi-agent systems to execute background troubleshooting checks when a specific technical tag is generated.
- **Live Notifications:** Building active email or messaging webhooks to ping relevant engineering personnel the moment a High priority tier is triggered.
```